

index.html

/mnt/user-data/outputs/index.html

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>B-T AI – chat</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&family=Roboto+Mono:wght@400;500&display=swap" rel="stylesheet">
<style>
:root{
  --bg-deep:#131314;
  --bg-panel:#1E1F20;
  --glass:rgba(255,255,255,0.045);
  --glass-hover:rgba(255,255,255,0.08);
  --glass-border:rgba(255,255,255,0.09);
  --ink:#E3E3E3;
  --ink-soft:#9AA0A6;
  --violet:#8AB4F8;
  --cyan:#AECBFA;
  --accent-grad:linear-gradient(135deg,var(--violet),var(--cyan));
  --line:rgba(255,255,255,0.09);
  --danger:#FFB6B6;
  --radius:14px;
}
*{box-sizing:border-box;margin:0;padding:0;}
html,body{height:100%;}
body{
  font-family:'Roboto','Google Sans Text',sans-serif;
  background:var(--bg-deep);
  color:var(--ink);
  display:flex;
  height:100vh;
  overflow:hidden;
  position:relative;
}
::selection{background:rgba(138,180,248,0.35);}

/* Ambient glow orbs – signature element */
.glow-orb{display:none;}
.glow-orb.o1{width:520px;height:520px;background:var(--violet);top:-160px;left:-120px;animation:drift1 22s ease-in-out infinite;}
.glow-orb.o2{width:460px;height:460px;background:var(--cyan);bottom:-140px;right:-100px;animation:drift2 26s ease-in-out infinite;}
.glow-orb.o3{width:340px;height:340px;background:var(--violet);top:40%;left:45%;opacity:.16;animation:drift3 30s ease-in-out infinite;}
@keyframes drift1{0%,100%{transform:translate(0,0);}50%{transform:translate(60px,40px);}}
@keyframes drift2{0%,100%{transform:translate(0,0);}50%{transform:translate(-50px,-30px);}}
@keyframes drift3{0%,100%{transform:translate(0,0) scale(1);}50%{transform:translate(-40px,30px) scale(1.15);}}
@media (prefers-reduced-motion: reduce){.glow-orb{animation:none;}}

.sidebar{
  width:270px;
  min-width:270px;
  background:var(--glass);
  backdrop-filter:blur(28px);
  -webkit-backdrop-filter:blur(28px);
  border-right:1px solid var(--glass-border);
  color:var(--ink);
  display:flex;
  flex-direction:column;
  padding:20px 16px;
  position:relative;
  z-index:2;
}
.brand{display:flex;align-items:baseline;gap:8px;margin-bottom:22px;padding:0 4px;}
.brand .mark{
  font-family:'Google Sans Text','Roboto',sans-serif;font-size:21px;font-weight:700;
  background:var(--accent-grad);-webkit-background-clip:text;background-clip:text;color:transparent;
}
.brand .sub{font-family:'Roboto Mono',monospace;font-size:10px;letter-spacing:.08em;color:var(--ink-soft);text-transform:uppercase;}
.nav-item{
  display:flex;align-items:center;gap:14px;padding:10px 10px;border-radius:10px;
```

```

    font-size:14px;color:var(--ink);cursor:pointer;transition:background .16s ease;
}
.nav-item:hover{background:var(--glass-hover);}
.nav-item svg{width:17px;height:17px;flex-shrink:0;color:var(--ink-soft);}
.nav-item span{white-space:nowrap;}

.search-nav{position:relative;}
.search-nav input{
    flex:1;min-width:0;border:none;outline:none;background:transparent;color:var(--ink);
    font-size:14px;font-family:inherit;cursor:text;
}
.search-nav input::placeholder{color:var(--ink);}
.search-clear{
    display:none;width:16px;height:16px;border-radius:50%;background:rgba(255,255,255,0.12);
    align-items:center;justify-content:center;cursor:pointer;color:var(--ink-soft);flex-shrink:0;
    font-size:10px;line-height:1;transition:background .15s ease;
}
.search-clear:hover{background:rgba(255,255,255,0.22);color:var(--ink);}
.search-clear.show{display:flex;}

.recents-label{
    margin:18px 4px 6px;font-family:'Roboto Mono',monospace;font-size:10.5px;
    letter-spacing:.06em;text-transform:uppercase;color:var(--ink-soft);
}
.thread-list{flex:1;overflow-y:auto;display:flex;flex-direction:column;gap:1px;}

.profile-row{
    position:relative;margin-top:12px;padding-top:12px;border-top:1px solid var(--line);
    display:flex;align-items:center;gap:10px;cursor:pointer;padding:10px;border-radius:10px;
    transition:background .16s ease;
}
.profile-row:hover{background:var(--glass-hover);}
.profile-avatar{
    width:30px;height:30px;border-radius:50%;background:var(--accent-grad);color:#0A0B12;
    display:flex;align-items:center;justify-content:center;font-family:'Google Sans Text','Roboto',sans-serif;
    font-size:13px;font-weight:700;flex-shrink:0;
}
.profile-name{flex:1;min-width:0;font-size:13.5px;color:var(--ink);overflow:hidden;text-overflow:ellipsis;white-space:nowrap;}
.profile-gear{width:20px;height:20px;color:var(--ink-soft);flex-shrink:0;display:flex;align-items:center;justify-content:center;}
.profile-gear svg{width:16px;height:16px;}
.profile-menu{
    display:none;position:absolute;bottom:calc(100% + 6px);left:10px;right:10px;
    background:#14151F;border:1px solid var(--glass-border);border-radius:12px;padding:6px;
    box-shadow:0 12px 30px rgba(0,0,0,0.5);z-index:15;
}
.profile-menu.open{display:block;}
.profile-menu-item{
    padding:9px 12px;border-radius:8px;font-size:13px;color:var(--danger);cursor:pointer;
    transition:background .15s ease;
}
.profile-menu-item:hover{background:rgba(255,107,107,0.1);}

.thread-item{
    position:relative;padding:9px 34px 9px 12px;border-radius:8px;font-size:13.5px;color:var(--ink-soft);
    cursor:pointer;display:flex;align-items:center;
    transition:background .18s ease, transform .18s ease, color .18s ease;
}
.thread-item:hover{background:var(--glass-hover);transform:translateX(2px);color:var(--ink);}
.thread-item.active .thread-dots{display:flex;}
.thread-dots svg{width:14px;height:14px;}
.thread-menu{
    position:absolute;right:4px;top:calc(100% + 2px);z-index:15;min-width:130px;
    background:#14151F;border:1px solid var(--glass-border);border-radius:10px;padding:5px;
    box-shadow:0 12px 30px rgba(0,0,0,0.5);display:none;flex-direction:column;gap:1px;
}
.thread-menu.open{display:flex;}
.thread-menu-item{
    padding:7px 10px;border-radius:7px;font-size:12.5px;color:var(--ink);cursor:pointer;
    display:flex;align-items:center;gap:7px;transition:background .15s ease;
}

```

```

}
.thread-menu-item:hover{background:var(--glass-hover);}
.thread-menu-item.danger{color:var(--danger);}
.thread-menu-item svg{width:13px;height:13px;flex-shrink:0;}

.usage-caption{
  margin-top:8px;padding:0 4px;font-family:'Roboto Mono',monospace;font-size:10.5px;color:var(--ink-soft);
  display:flex;align-items:center;justify-content:space-between;
}
.usage-caption .bar{flex:1;height:3px;background:rgba(255,255,255,0.08);border-radius:99px;overflow:hidden;margin:0 8px;}
.usage-caption .bar-fill{height:100%;background:var(--accent-grad);border-radius:99px;transition:width .3s ease;}

.main{flex:1;display:flex;flex-direction:column;min-width:0;position:relative;z-index:1;}
.topbar{padding:14px 28px;border-bottom:1px solid var(--line);display:flex;justify-content:space-between;align-items:center;background:rgba(9,10,17,0.4);backdrop-filter:blur(16px);}
.topbar-title{font-family:'Google Sans Text','Roboto',sans-serif;font-size:14.5px;font-weight:600;color:var(--ink);}
.topbar-title span{color:var(--ink-soft);font-weight:400;font-family:'Roboto','Google Sans Text',sans-serif;}
.model-pill{font-family:'Roboto Mono',monospace;font-size:11px;padding:4px 10px;border:1px solid var(--glass-border);background:var(--glass);border-radius:99px;color:var(--ink-soft);}

.messages{flex:1;overflow-y:auto;padding:32px 0 12px;}
.msg-wrap{
  max-width:680px;margin:0 auto 18px;padding:0 24px;display:flex;
  animation:msgIn .35s ease both;
}
.msg-wrap.user{justify-content:flex-end;}
.msg-wrap.ai{justify-content:flex-start;}
.msg-col{display:flex;flex-direction:column;max-width:78%;}
.msg-wrap.ai .msg-col{max-width:100%;}
.msg-wrap.user .msg-col{align-items:flex-end;}
.msg-wrap.ai .msg-col{align-items:flex-start;}
.msg-col .bubble{max-width:100%;}
@keyframes msgIn{
  from{opacity:0;transform:translateY(8px);}
  to{opacity:1;transform:translateY(0);}
}
.bubble{
  max-width:100%;padding:10px 14px;border-radius:16px;position:relative;
}
.msg-wrap.user .bubble{
  background:var(--violet);color:#131314;border-bottom-right-radius:5px;
}
.msg-wrap.user .bubble::after{
  content:'';position:absolute;bottom:0;right:-6px;width:12px;height:14px;
  background:var(--violet);
  clip-path:polygon(0 0, 0% 100%, 100% 100%);
}
.msg-wrap.ai .bubble{
  background:transparent;border:none;color:var(--ink);padding:0;
}
.bubble .text{font-size:15px;line-height:1.55;white-space:pre-wrap;}
.msg-wrap.user .bubble .text{color:#131314;}
.code-block{background:rgba(0,0,0,0.35);border:1px solid var(--glass-border);border-radius:8px;padding:10px 12px;margin:6px 0;overflow-x:auto;font-family:'Roboto Mono',monospace;font-size:13px;}
.inline-code{background:rgba(0,0,0,0.3);border-radius:4px;padding:1px 5px;font-family:'Roboto Mono',monospace;font-size:13px;}
.md-list{margin:4px 0 4px 18px;}
.md-list li{margin-bottom:2px;}
.msg-meta{display:flex;align-items:center;gap:8px;margin-top:5px;padding:0 4px;opacity:.65;}
.msg-time{font-family:'Roboto Mono',monospace;font-size:10.5px;color:var(--ink-soft);}
.msg-wrap.user .msg-time{color:var(--ink-soft);}
.msg-action{width:20px;height:20px;display:flex;align-items:center;justify-content:center;border-radius:6px;cursor:pointer;color:var(--ink-soft);transition:all .15s ease;}
.msg-action.active-up{color:var(--violet);}
.msg-action.active-down{color:var(--danger);}
.msg-action:hover{background:rgba(255,255,255,0.1);color:var(--ink);}
.msg-action svg{width:13px;height:13px;}
.send-btn.stop-mode{background:var(--danger);color:#fff;}
.send-btn.stop-mode:hover{background:var(--danger);box-shadow:0 0 18px rgba(255,107,107,0.5);}
.image-result{position:relative;display:inline-block;margin-top:8px;max-width:100%;}
.gen-image{max-width:100%;border-radius:14px;display:block;border:1px solid var(--glass-border);box-shadow:0 0 30px rgba(138,180,248,0.15);}
.download-btn{
  position:absolute;top:10px;right:10px;width:34px;height:34px;border-radius:9px;
  background:rgba(9,10,17,0.6);backdrop-filter:blur(8px);border:1px solid var(--glass-border);
  display:flex;align-items:center;justify-content:center;cursor:pointer;text-decoration:none;
  transition:all .2s ease;
}

```

```

.download-btn:hover{background:var(--accent-grad);box-shadow:0 0 16px rgba(138,180,248,0.5);}
.download-btn svg{width:16px;height:16px;color:#fff;}
.attach-chip{
  display:flex;align-items:center;gap:8px;max-width:680px;margin:0 auto 8px;padding:6px 10px;
  background:var(--glass);border:1px solid var(--glass-border);border-radius:10px;font-size:12.5px;color:var(--
ink-soft);
}
.attach-chip img{width:28px;height:28px;border-radius:6px;object-fit:cover;}
.attach-chip .remove{margin-left:auto;cursor:pointer;color:var(--ink-soft);font-size:16px;line-height:1;}
.attach-chip .remove:hover{color:var(--danger);}
.user-attach-img{max-width:220px;border-radius:12px;margin-top:6px;display:block;border:1px solid var(--glass-b
order);}
.mode-chip-row{max-width:680px;margin:0 auto 8px;display:flex;}
.mode-chip{
  display:flex;align-items:center;gap:8px;padding:6px 12px;border-radius:99px;
  background:var(--accent-grad);color:#0A0B12;font-size:12.5px;font-weight:600;
  box-shadow:0 0 16px rgba(138,180,248,0.35);
}
.mode-chip .x{cursor:pointer;font-weight:700;opacity:.75;}
.mode-chip .x:hover{opacity:1;}

.empty-state{max-width:520px;margin:10vh auto 0;text-align:center;padding:0 24px;animation:msgIn .4s ease both;
}
.empty-state .mark{font-family:'Google Sans Text','Roboto',sans-serif;font-size:34px;font-weight:700;background
:var(--accent-grad);-webkit-background-clip:text;background-clip:text;color:transparent;margin-bottom:10px;}
.empty-state p{color:var(--ink-soft);font-size:14.5px;line-height:1.6;}

.composer{padding:16px 24px 24px;}
.composer-inner{
  max-width:680px;margin:0 auto;display:flex;align-items:flex-end;gap:8px;
  background:rgba(255,255,255,0.055);backdrop-filter:blur(26px);-webkit-backdrop-filter:blur(26px);
  border:1px solid var(--glass-border);border-radius:22px;padding:9px 9px 9px 18px;
  box-shadow:0 8px 28px rgba(0,0,0,0.28), inset 0 1px 0 rgba(255,255,255,0.04);
  transition:border-color .28s cubic-bezier(.2,.8,.2,1), box-shadow .28s cubic-bezier(.2,.8,.2,1);
}
.composer-inner:focus-within{
  border-color:rgba(138,180,248,0.55);
  box-shadow:0 8px 28px rgba(0,0,0,0.28), 0 0 0 1px rgba(138,180,248,0.18), 0 0 36px rgba(138,180,248,0.22), 0
64px rgba(174,203,250,0.09);
}
.textarea{flex:1;border:none;outline:none;resize:none;font-family:'Roboto','Google Sans Text',sans-serif;font-si
ze:15px;line-height:1.5;max-height:160px;color:var(--ink);background:transparent;padding:7px 0;}
.textarea::placeholder{color:var(--ink-soft);}
.send-btn{
  width:38px;height:38px;border-radius:50%;background:rgba(255,255,255,0.07);border:1px solid var(--glass-borde
r);color:var(--ink);
  display:flex;align-items:center;justify-content:center;cursor:pointer;flex-shrink:0;
  transition:background .2s cubic-bezier(.2,.8,.2,1), transform .18s cubic-bezier(.34,1.56,.64,1), box-shadow .
2s ease;
}
.send-btn:hover:not(:disabled){background:var(--accent-grad);color:#0A0B12;box-shadow:0 0 18px rgba(138,180,248
,0.5);transform:scale(1.08);}
.send-btn:active:not(:disabled){transform:scale(0.94);}
.send-btn:disabled{background:var(--glass);color:var(--ink-soft);cursor:not-allowed;opacity:.5;}
.send-btn svg{width:16px;height:16px;}
.image-btn{
  width:38px;height:38px;border-radius:50%;background:rgba(255,255,255,0.07);border:1px solid var(--glass-borde
r);color:var(--ink);
  display:flex;align-items:center;justify-content:center;cursor:pointer;flex-shrink:0;font-size:16px;
  transition:background .2s cubic-bezier(.2,.8,.2,1), transform .18s cubic-bezier(.34,1.56,.64,1), box-shadow .
2s ease;
}
.image-btn:hover{background:var(--accent-grad);box-shadow:0 0 18px rgba(174,203,250,0.4);transform:scale(1.08);}
.image-btn:active{transform:scale(0.94);}
.image-btn svg{width:16px;height:16px;}
.image-btn.mic-active{background:transparent;box-shadow:none;}

.voice-indicator{
  display:none;align-items:center;justify-content:center;gap:4px;
  max-width:680px;margin:0 auto 8px;padding:10px 20px;
  background:var(--glass);border:1px solid var(--glass-border);border-radius:99px;
  backdrop-filter:blur(20px);-webkit-backdrop-filter:blur(20px);
  box-shadow:0 0 24px rgba(138,180,248,0.25);
}
.voice-indicator.active{display:flex;}
.voice-indicator .bar{
  width:4px;border-radius:99px;background:var(--accent-grad);
  box-shadow:0 0 8px rgba(138,180,248,0.6);
  animation:voiceBar 1s ease-in-out infinite;
}

```

```

.voice-indicator .bar:nth-child(1){height:10px;animation-delay:0s;}
.voice-indicator .bar:nth-child(2){height:20px;animation-delay:.1s;}
.voice-indicator .bar:nth-child(3){height:28px;animation-delay:.2s;}
.voice-indicator .bar:nth-child(4){height:18px;animation-delay:.3s;}
.voice-indicator .bar:nth-child(5){height:24px;animation-delay:.15s;}
.voice-indicator .bar:nth-child(6){height:14px;animation-delay:.25s;}
.voice-indicator .bar:nth-child(7){height:10px;animation-delay:.05s;}
@keyframes voiceBar{
  0%,100%{transform:scaleY(0.4);}
  50%{transform:scaleY(1);}
}
.voice-indicator .label{margin-left:10px;font-size:12.5px;color:var(--ink-soft);font-family:'Roboto Mono',monospace;}
@media (prefers-reduced-motion: reduce){.voice-indicator .bar{animation:none;}}

.plus-menu-wrap{position:relative;}
.plus-menu{
  position:absolute;bottom:calc(100% + 10px);left:0;min-width:190px;
  background:#14151F;backdrop-filter:blur(24px);-webkit-backdrop-filter:blur(24px);
  border:1px solid var(--glass-border);border-radius:14px;padding:6px;
  box-shadow:0 12px 40px rgba(0,0,0,0.5), 0 0 24px rgba(138,180,248,0.1);
  display:none;flex-direction:column;gap:2px;z-index:20;
}
.plus-menu.open{display:flex;animation:menuIn .16s ease both;}
@keyframes menuIn{from{opacity:0;transform:translateY(4px);}to{opacity:1;transform:translateY(0);}}
.plus-menu-item{
  display:flex;align-items:center;gap:10px;padding:9px 10px;border-radius:9px;
  font-size:13.5px;color:var(--ink);cursor:pointer;transition:background .15s ease;
}
.plus-menu-item:hover{background:var(--glass-hover);}
.plus-menu-item svg{width:16px;height:16px;flex-shrink:0;color:var(--ink-soft);}
.composer-note{text-align:center;font-size:11px;color:var(--ink-soft);margin-top:10px;font-family:'Roboto Mono',monospace;}

.typing{display:flex;gap:5px;padding:8px 2px;}
.typing span{
  width:8px;height:8px;border-radius:50%;
  animation:googleBounce 1.4s ease-in-out infinite;
}
.typing span:nth-child(1){background:#4285F4;animation-delay:0s;}
.typing span:nth-child(2){background:#EA4335;animation-delay:.15s;}
.typing span:nth-child(3){background:#FBBC05;animation-delay:.3s;}
.typing span:nth-child(4){background:#34A853;animation-delay:.45s;}
@keyframes googleBounce{
  0%,60%,100%{transform:translateY(0);opacity:.6;}
  30%{transform:translateY(-7px);opacity:1;}
}
@media (prefers-reduced-motion: reduce){.typing span{animation:none;}.glow-orb{animation:none;}}
.menu-toggle{
  display:none;width:34px;height:34px;border-radius:9px;background:var(--glass);
  border:1px solid var(--glass-border);color:var(--ink);align-items:center;justify-content:center;
  cursor:pointer;flex-shrink:0;margin-right:6px;transition:all .2s ease;
}
.menu-toggle svg{width:18px;height:18px;}
.menu-toggle:hover{background:var(--glass-hover);}
.sidebar-overlay{
  display:none;position:fixed;inset:0;background:rgba(0,0,0,0.5);z-index:9;
  backdrop-filter:blur(2px);opacity:0;transition:opacity .2s ease;
}
.sidebar-overlay.open{display:block;opacity:1;}

.call-screen{
  display:none;position:fixed;inset:0;z-index:100;
  background:radial-gradient(circle at 50% 30%, #171226 0%, #090A11 70%);
  flex-direction:column;align-items:center;justify-content:center;gap:36px;
}
.call-screen.open{display:flex;animation:callFadeIn .35s ease both;}
@keyframes callFadeIn{from{opacity:0;}to{opacity:1;}}

.call-orb-wrap{position:relative;width:220px;height:220px;display:flex;align-items:center;justify-content:center;}
.call-orb{
  width:150px;height:150px;border-radius:50%;
  background:radial-gradient(circle at 35% 30%, #A78BFA, #8B5CF6 45%, #22D3EE 100%);
  box-shadow:0 0 60px rgba(138,180,248,0.55), 0 0 120px rgba(174,203,250,0.25);
  animation:orbBreathe 3.2s ease-in-out infinite;
}
.call-orb-ring{
  position:absolute;width:220px;height:220px;border-radius:50%;
  border:1px solid rgba(138,180,248,0.3);
  animation:ringPulse 3.2s ease-in-out infinite;
}

```

```

}
@keyframes orbBreathe{
  0%,100%{transform:scale(1);}
  50%{transform:scale(1.08);}
}
@keyframes ringPulse{
  0%{transform:scale(0.85);opacity:.6;}
  100%{transform:scale(1.25);opacity:0;}
}
.call-screen.listening .call-orb{animation:orbBreathe 1.6s ease-in-out infinite;}
.call-screen.listening .call-orb-ring{animation:ringPulse 1.6s ease-in-out infinite;}
.call-screen.thinking .call-orb{animation:orbThink 1s linear infinite;}
@keyframes orbThink{
  0%{transform:scale(1) rotate(0deg);}
  50%{transform:scale(0.94) rotate(180deg);}
  100%{transform:scale(1) rotate(360deg);}
}
.call-screen.speaking .call-orb{animation:orbSpeak .5s ease-in-out infinite;}
@keyframes orbSpeak{
  0%,100%{transform:scale(1);}
  50%{transform:scale(1.15);}
}
@media (prefers-reduced-motion: reduce){
  .call-orb, .call-orb-ring{animation:none !important;}
}

.call-status{font-family:'Google Sans Text','Roboto',sans-serif;font-size:16px;font-weight:500;color:var(--ink-soft);letter-spacing:.02em;}

.call-controls{display:flex;gap:22px;}
.call-btn{
  width:58px;height:58px;border-radius:50%;border:none;cursor:pointer;
  display:flex;align-items:center;justify-content:center;transition:all .2s ease;
}
.call-btn.svg{width:24px;height:24px;}
.call-btn.mute{background:rgba(255,255,255,0.08);border:1px solid var(--glass-border);color:var(--ink);}
.call-btn.mute:hover{background:rgba(255,255,255,0.14);}
.call-btn.mute.muted{background:rgba(255,255,255,0.9);color:#0A0B12;}
.call-btn.end{background:var(--danger);color:#fff;box-shadow:0 0 20px rgba(255,107,107,0.4);}
.call-btn.end:hover{filter:brightness(1.1);transform:scale(1.05);}

@media (max-width:720px){
  .sidebar{position:fixed;z-index:10;height:100%;transform:translateX(-100%);transition:transform .22s ease;}
  .sidebar.open{transform:translateX(0);}
  .menu-toggle{display:flex;}
  .topbar{padding:12px 16px;}
  .messages{padding:20px 0 12px;}
  .msg-wrap{padding:0 14px;}
  .composer{padding:12px 14px 16px;}
  .empty-state{padding:0 16px;}
}
</style>
</head>
<body>

<div class="glow-orb o1"></div>
<div class="glow-orb o2"></div>
<div class="glow-orb o3"></div>

<div class="sidebar" id="sidebar">
  <div class="brand">
    <span class="mark">B-T AI</span>
  </div>

  <div class="nav-item" id="newChatBtn">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 5v14M5 12h14"/></svg>
    <span>New chat</span>
  </div>
  <div class="nav-item search-nav" id="searchNavToggle">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="11" cy="11" r="7"/></svg>
    <input type="text" id="threadSearch" placeholder="Search chats">
    <span class="search-clear" id="searchClear">X</span>
  </div>

  <div class="nav-item" id="upgradeBtn">
    <svg viewBox="0 0 24 24" fill="currentColor"><path d="M12 2l2.4 7.2L22 12l-7.6 2.8L12 22l-2.4-7.2L2 12l7.6-2.8L12 2z"/></svg>
    <span>Upgrade</span>
  </div>

```

```

<div class="recents-label">Recents</div>
<div class="thread-list" id="threadList"></div>

<div class="usage-caption">
  <span id="planCaption">0 / 100</span>
  <span class="bar"><span class="bar-fill" id="planMeterFill" style="width:0%"></span></span>
  <span>today</span>
</div>

<div class="profile-row" id="profileRow">
  <div class="profile-avatar" id="profileAvatar">B</div>
  <div class="profile-name" id="userGreeting">Account</div>
  <div class="profile-gear" id="profileGear" title="Settings">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="3" /><path d="M19.4 15a1.65 1.65 0 00-.33 1.82l-.06.06a2 2 0 11-2.83 2.83l-.06-.06a1.65 1.65 0 00-1.82-.33 1.65 1.65 0 00-1.51v21a2 2 0 01-.4 0v-.09a1.65 1.65 0 009 19.4a1.65 1.65 0 00-1.82.33l-.06.06a2 2 0 11-2.83-2.83l-.06-.06a1.65 1.65 0 004.6 15a1.65 1.65 0 00-1.51-1H3a2 2 0 01-4h.09A1.65 1.65 0 004.6 9a1.65 1.65 0 00-.33-1.82l-.06-.06a2 2 0 112.83-2.83l-.06.06A1.65 1.65 0 009 4.6a1.65 1.65 0 001-1.51V3a2 2 0 014 0v.09a1.65 1.65 0 001 1.51 1.65 1.65 0 001.82-.33l-.06-.06a2 2 0 112.83 2.83l-.06.06A1.65 1.65 0 0019.4 9a1.65 1.65 0 001.51 1H21a2 2 0 010 4h-.09a1.65 1.65 0 00-1.51 1z"/></svg>
  </div>
  <div class="profile-menu" id="profileMenu">
    <div class="profile-menu-item" id="logoutLink">Log out</div>
  </div>
</div>
<div class="sidebar-overlay" id="sidebarOverlay"></div>

<div class="call-screen" id="callScreen">
  <div class="call-orb-wrap">
    <div class="call-orb-ring"></div>
    <div class="call-orb" id="callOrb"></div>
  </div>
  <div class="call-status" id="callStatus">Listening...</div>
  <div class="call-controls">
    <button class="call-btn mute" id="callMuteBtn" title="Mute">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 1a3 3 0 00-3 3v8a3 3 0 006 0V4a3 3 0 00-3 3z"/><path d="M19 10v2a7 7 0 01-14 0v-2M12 19v4M8 23h8"/></svg>
    </button>
    <button class="call-btn end" id="callEndBtn" title="End call">
      <svg viewBox="0 0 24 24" fill="currentColor"><path d="M12 9c-2.5 0-4.95-7 1.4a1 1 0 00-.6 1v2.6a1 1 0 00 1 1c1.3 0 2.5-.2 3.7-.6a1 1 0 011 .2l1.6 1.6a1 1 0 001.4 0l1.6-1.6a1 1 0 011-.2c1.2 4 2.4 6 3.7 6a1 1 0 001-1v-2.6a1 1 0 00-.6-1A17.6 17.6 0 0012 9z" transform="rotate(135 12 12)"/></svg>
    </button>
  </div>
</div>

<div class="main">
  <div class="topbar">
    <button class="menu-toggle" id="menuToggle" aria-label="Menu">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 6h18M3 12h18M3 18h18"/></svg>
    </button>
    <div class="topbar-title" id="threadTitle">New chat</div>
    <div class="model-pill">B-T AI</div>
  </div>
  <div class="messages" id="messages">
    <div class="empty-state" id="emptyState">
      <div class="mark">B-T AI</div>
      <p>Type a question below to start chatting.</p>
    </div>
  </div>
  <div class="composer">
    <div class="voice-indicator" id="voiceIndicator">
      <span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="bar"></span><span class="label">Listening...</span>
    </div>
    <div class="mode-chip-row" id="modeChipRow"></div>
    <div class="attach-preview" id="attachPreview"></div>
    <div class="composer-inner">
      <input type="file" id="fileInput" accept="image/*" style="display:none;">
      <div class="plus-menu-wrap">
        <button class="image-btn" id="attachBtn" title="Add" aria-label="Add">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 5v14M5 12h14" /></svg>
        </button>
        <div class="plus-menu" id="plusMenu">
          <div class="plus-menu-item" id="menuUploadFiles">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0

```

```

01-2 2H5a2 2 0 01-2-2v-4M17 8l-5-5-5 5M12 3v12"/></svg>
    <span>Upload files</span>
  </div>
  <div class="plus-menu-item" id="menuCreateImage">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x="3" y="3" width
=18" height="18" rx="2"/><circle cx="8.5" cy="8.5" r="1.5"/><path d="M21 15l-5-5L5 21"/></svg>
    <span>Create image</span>
  </div>
</div>
</div>
<div class="input" rows="1" placeholder="Ask B-T AI"></div>
<div class="image-btn" id="micBtn" title="Voice input" aria-label="Voice input">
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 1a3 3 0 0-3 3
v8a3 3 0 006 0V4a3 3 0 00-3-3z"/><path d="M19 10v2a7 7 0 01-14 0v-2M12 19v4M8 23h8"/></svg>
</div>
<div class="send-btn" id="sendBtn" aria-label="Voice input">
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 8v8M8 5v14M12 3
v18M16 5v14M20 8v8"/></svg>
</div>
<div class="composer-note">B-T AI can make mistakes. Check important info.</div>
</div>
</div>

<script>
// Always talk to the Bonto backend directly, no matter which domain
// (Bonto preview or GitHub Pages) is currently serving this HTML file.
const BACKEND_URL = 'https://bachakhanb-tai.bonto.run';

let currentUserId = localStorage.getItem('bt_user_id');
let currentUserName = localStorage.getItem('bt_user_name');
if (!currentUserId) {
  currentUserId = 'demo-user';
  currentUserName = 'Demo';
}
document.getElementById('userGreeting').textContent = currentUserName || 'Account';
document.getElementById('profileAvatar').textContent = (currentUserName || 'B').trim().charAt(0).toUpperCase();

document.getElementById('logoutLink').onclick = () => {
  localStorage.removeItem('bt_user_id');
  localStorage.removeItem('bt_user_name');
  window.location.href = 'auth.html';
};

const profileGear = document.getElementById('profileGear');
const profileMenu = document.getElementById('profileMenu');
profileGear.addEventListener('click', (e) => {
  e.stopPropagation();
  profileMenu.classList.toggle('open');
});
document.addEventListener('click', (e) => {
  if(!profileMenu.contains(e.target) && e.target !== profileGear){
    profileMenu.classList.remove('open');
  }
});

const DAILY_LIMIT = 100;
const state = {
  threads: JSON.parse(localStorage.getItem('bt_threads')) || '[]',
  activeId: null,
  usedToday: parseInt(localStorage.getItem('bt_used_' + todayKey())) || '0', 10),
};

function todayKey(){ return new Date().toISOString().slice(0,10); }
function save(){ localStorage.setItem('bt_threads', JSON.stringify(state.threads)); }
function saveUsage(){ localStorage.setItem('bt_used_' + todayKey(), String(state.usedToday)); }

const threadListEl = document.getElementById('threadList');
const messagesEl = document.getElementById('messages');
const emptyStateEl = document.getElementById('emptyState');
const inputEl = document.getElementById('input');
const sendBtn = document.getElementById('sendBtn');
const attachBtn = document.getElementById('attachBtn');
const fileInput = document.getElementById('fileInput');
const attachPreview = document.getElementById('attachPreview');
const micBtn = document.getElementById('micBtn');
const threadTitleEl = document.getElementById('threadTitle');
const planMeterFill = document.getElementById('planMeterFill');
const planCaption = document.getElementById('planCaption');

const sidebarEl = document.getElementById('sidebar');

```



```

const sidebarOverlay = document.getElementById('sidebarOverlay');
const menuToggle = document.getElementById('menuToggle');

function openSidebar(){
  sidebarEl.classList.add('open');
  sidebarOverlay.classList.add('open');
}
function closeSidebar(){
  sidebarEl.classList.remove('open');
  sidebarOverlay.classList.remove('open');
}
menuToggle.addEventListener('click', () => {
  sidebarEl.classList.contains('open') ? closeSidebar() : openSidebar();
});
sidebarOverlay.addEventListener('click', closeSidebar);

const threadSearch = document.getElementById('threadSearch');
const searchNavToggle = document.getElementById('searchNavToggle');
searchNavToggle.addEventListener('click', (e) => {
  if(e.target.closest('.search-clear')) return;
  threadSearch.focus();
});
const searchClear = document.getElementById('searchClear');
let searchQuery = '';
threadSearch.addEventListener('input', () => {
  searchQuery = threadSearch.value.trim().toLowerCase();
  searchClear.classList.toggle('show', threadSearch.value.length > 0);
  renderThreadList();
});
searchClear.addEventListener('click', () => {
  threadSearch.value = '';
  searchQuery = '';
  searchClear.classList.remove('show');
  renderThreadList();
  threadSearch.focus();
});

function renderThreadList(){
  threadListEl.innerHTML = '';
  state.threads
    .slice().reverse()
    .filter(t => t.title.toLowerCase().includes(searchQuery))
    .forEach(t => {
      const item = document.createElement('div');
      item.className = 'thread-item' + (t.id === state.activeId ? ' active' : '');
      item.innerHTML = `
        <span class="thread-title-text"></span>
        <div class="thread-dots" title="More">
          <svg viewBox="0 0 24 24" fill="currentColor"><circle cx="5" cy="12" r="1.6"/><circle cx="12" cy="12"
r="1.6"/><circle cx="19" cy="12" r="1.6"/></svg>
        </div>
        <div class="thread-menu">
          <div class="thread-menu-item" data-action="rename">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4h6M7 20h1
0M9 4L4 20M15 4L5 20" /></svg>
            Rename
          </div>
          <div class="thread-menu-item danger" data-action="delete">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 6h18M8 6V4a
2 2 0 0 12-2h4a2 2 0 0 12 2v2m3 0l-1 14a2 2 0 0 1-2 2H7a2 2 0 0 1-2 2L4 6"/></svg>
            Delete
          </div>
        </div>`;
      item.querySelector('.thread-title-text').textContent = t.title;

      item.onclick = (e) => {
        if(e.target.closest('.thread-dots') || e.target.closest('.thread-menu')) return;
        state.activeId = t.id;
        renderAll();
        closeSidebar();
      };

      const dots = item.querySelector('.thread-dots');
      const menu = item.querySelector('.thread-menu');
      dots.onclick = (e) => {
        e.stopPropagation();
        document.querySelectorAll('.thread-menu.open').forEach(m => { if(m !== menu) m.classList.remove('open')
; });
        menu.classList.toggle('open');
      };
    });
}

```

```

    menu.querySelector('[data-action="rename"]').onclick = (e) => {
      e.stopPropagation();
      menu.classList.remove('open');
      const newTitle = window.prompt('Rename chat:', t.title);
      if(newTitle && newTitle.trim()){
        t.title = newTitle.trim().slice(0, 60);
        save();
        renderAll();
      }
    };
    menu.querySelector('[data-action="delete"]').onclick = (e) => {
      e.stopPropagation();
      menu.classList.remove('open');
      if(confirm(`Delete "${t.title}"? This can't be undone.`)){
        state.threads = state.threads.filter(x => x.id !== t.id);
        if(state.activeId === t.id) state.activeId = null;
        save();
        renderAll();
      }
    };
    threadListEl.appendChild(item);
  });
}

document.addEventListener('click', () => {
  document.querySelectorAll('.thread-menu.open').forEach(m => m.classList.remove('open'));
});

function escapeHtml(str){
  return str.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
}

function formatMarkdown(text){
  let t = escapeHtml(text || '');
  t = t.replace(/``([\s\S]*)``/g, (m, code) => `

```
<code>${code.trim()}</code></pre>`);
 t = t.replace(/\`([\^]+)\`/g, '<code class="inline-code">${1}</code>');
 t = t.replace(/**([\^]+)**/g, '${1}');
 const lines = t.split('\n');
 let html = '';
 let inList = false;
 lines.forEach(line => {
 const bulletMatch = line.match(/^[-*]\s+(.*)/);
 if(bulletMatch){
 if(!inList){ html += '<ul class="md-list">'; inList = true; }
 html += '${bulletMatch[1]}';
 } else {
 if(inList){ html += ''; inList = false; }
 html += line + '\n';
 }
 });
 if(inList) html += '';
 return html;
}

function formatTime(ts){
 if(!ts) return '';
 return new Date(ts).toLocaleTimeString([], { hour: 'numeric', minute: '2-digit' });
}

function renderMessages(){
 const thread = state.threads.find(t => t.id === state.activeId);
 messagesEl.innerHTML = '';
 if(!thread || thread.messages.length === 0){
 messagesEl.appendChild(emptyStateEl);
 threadTitleEl.textContent = 'New chat';
 return;
 }
 threadTitleEl.textContent = thread.title;
 thread.messages.forEach((m, idx) => {
 const wrap = document.createElement('div');
 wrap.className = 'msg-wrap ' + (m.role === 'user' ? 'user' : 'ai');
 wrap.innerHTML = '<div class="msg-col"><div class="bubble"><div class="text"></div></div></div>';
 const textEl = wrap.querySelector('.text');
 const bubbleEl = wrap.querySelector('.bubble');

 if(m.type === 'image'){
 if(m.imageUrl){
 textEl.textContent = m.content || '';
 const resultWrap = document.createElement('div');
 resultWrap.className = 'image-result';
 }
 }
 });
}

```


```

```

const img = document.createElement('img');
img.className = 'gen-image';
img.src = m.imageUrl;
img.alt = m.content || 'Generated image';
const dl = document.createElement('a');
dl.className = 'download-btn';
dl.href = m.imageUrl;
dl.download = (m.content || 'bt-ai-image').slice(0, 40).replace(/[^a-z0-9]/gi, '_') + '.jpg';
dl.target = '_blank';
dl.rel = 'noopener';
dl.title = 'Download image';
dl.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M1
2 3v12m0 0l-4-4m4 4l4-4M 21h16"/></svg>`;
resultWrap.appendChild(img);
resultWrap.appendChild(dl);
bubbleEl.appendChild(resultWrap);
} else {
textEl.textContent = m.content;
}
} else if(m.role === 'assistant'){
textEl.innerHTML = formatMarkdown(m.content);
} else {
textEl.textContent = m.content;
}
}

if(m.attachedImage){
const attImg = document.createElement('img');
attImg.className = 'user-attach-img';
attImg.src = m.attachedImage;
bubbleEl.appendChild(attImg);
}

// Meta row: timestamp + action buttons
const meta = document.createElement('div');
meta.className = 'msg-meta';
const timeSpan = document.createElement('span');
timeSpan.className = 'msg-time';
timeSpan.textContent = formatTime(m.timestamp);
meta.appendChild(timeSpan);

if(m.role === 'assistant' && m.type !== 'image'){
const thumbUp = document.createElement('span');
thumbUp.className = 'msg-action';
thumbUp.title = 'Good response';
thumbUp.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d=
"M14 9V5a3 3 0 00-3-3l-4 9v11h11.28a2 2 0 002-1.7l1.38-9a2 2 0 00-2-2.3H14z"/><path d="M7 22H4a2 2 0 01-2-2v-7a2
2 0 012-2h3"/></svg>`;
thumbUp.onclick = () => {
thumbUp.classList.toggle('active-up');
thumbDown.classList.remove('active-down');
};
meta.appendChild(thumbUp);

const thumbDown = document.createElement('span');
thumbDown.className = 'msg-action';
thumbDown.title = 'Bad response';
thumbDown.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M10 15v4a3 3 0 003 3l4-9V2H5.72a2 2 0 00-2 1.7l-1.38 9a2 2 0 002 2.3H10z"/><path d="M17 2h3a2 2 0 012-2v-7a2
0 01-2-2h-3"/></svg>`;
thumbDown.onclick = () => {
thumbDown.classList.toggle('active-down');
thumbUp.classList.remove('active-up');
};
meta.appendChild(thumbDown);

const copyBtn = document.createElement('span');
copyBtn.className = 'msg-action';
copyBtn.title = 'Copy';
copyBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x=
"9" y="9" width="12" height="12" rx="2"/><path d="M5 15V5a2 2 0 012-2h10"/></svg>`;
copyBtn.onclick = () => {
navigator.clipboard.writeText(m.content).then(() => {
copyBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><pat
h d="M20 6L9 17l-5-5"/></svg>`;
setTimeout(() => {
copyBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><r
ect x="9" y="9" width="12" height="12" rx="2"/><path d="M5 15V5a2 2 0 012-2h10"/></svg>`;
}, 1500);
});
};
meta.appendChild(copyBtn);

```

```

    if(m.failed){
      const retryBtn = document.createElement('span');
      retryBtn.className = 'msg-action';
      retryBtn.title = 'Retry';
      retryBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M1 4v6h6M23 20v-6h-6"/><path d="M20.49 9A9 9 0 005.64 5.64L1 10m22 4l-4.64 4.36A9 9 0 013.51 15"/></svg>`;
      retryBtn.onclick = () => retryMessage(idx);
      meta.appendChild(retryBtn);
    } else {
      const regenBtn = document.createElement('span');
      regenBtn.className = 'msg-action';
      regenBtn.title = 'Regenerate';
      regenBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M1 4v6h6M23 20v-6h-6"/><path d="M20.49 9A9 9 0 005.64 5.64L1 10m22 4l-4.64 4.36A9 9 0 013.51 15"/></svg>`;
      regenBtn.onclick = () => retryMessage(idx);
      meta.appendChild(regenBtn);
    }
  }
  wrap.querySelector('.msg-col').appendChild(meta);

  messagesEl.appendChild(wrap);
});
messagesEl.scrollTop = messagesEl.scrollHeight;
}

function renderPlan(){
  const pct = Math.min(100, (state.usedToday / DAILY_LIMIT) * 100);
  planMeterFill.style.width = pct + '%';
  planCaption.textContent = `${state.usedToday} / ${DAILY_LIMIT}`;
}

function renderAll(){ renderThreadList(); renderMessages(); renderPlan(); }

document.getElementById('newChatBtn').onclick = () => {
  const thread = { id: crypto.randomUUID(), title: 'New chat', messages: [] };
  state.threads.push(thread);
  state.activeId = thread.id;
  save();
  renderAll();
  closeSidebar();
  inputEl.focus();
};

document.getElementById('upgradeBtn').onclick = () => {
  alert('Connect your payment flow here (EasyPaisa / JazzCash / card) – the transaction will be recorded in the
payments table (schema.sql).');
};

const SEND_ICON = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 19V
5M5 12l7-7 7 7"/></svg>`;
const CALL_ICON = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 8v8M
8 5v14M12 3v18M16 5v14M20 8v8"/></svg>`;

function updateSendButtonIcon(){
  if(sendBtn.classList.contains('stop-mode')) return;
  const hasContent = inputEl.value.trim().length > 0 || !!pendingAttachment;
  sendBtn.disabled = false;
  if(hasContent){
    sendBtn.innerHTML = SEND_ICON;
    sendBtn.setAttribute('aria-label', 'Send');
    sendBtn.title = 'Send';
  } else {
    sendBtn.innerHTML = CALL_ICON;
    sendBtn.setAttribute('aria-label', 'Voice input');
    sendBtn.title = 'Voice input';
  }
}

inputEl.addEventListener('input', () => {
  updateSendButtonIcon();
  inputEl.style.height = 'auto';
  inputEl.style.height = Math.min(inputEl.scrollHeight, 160) + 'px';
});
inputEl.addEventListener('keydown', (e) => {
  if(e.key === 'Enter' && !e.shiftKey){ e.preventDefault(); sendMessage(); }
});
sendBtn.addEventListener('click', () => {
  if(sendBtn.classList.contains('stop-mode')){
    if(activeController) activeController.abort();
    return;
  }

```

```

    }
    const hasContent = inputEl.value.trim().length > 0 || !!pendingAttachment;
    if(hasContent){
        sendMessage();
    } else {
        openCallMode();
    }
});

// -----
// Attach / upload – like Gemini's attach button under the
// composer. Shows a small preview chip; the image travels
// with the next message the user sends.
// -----
let pendingAttachment = null; // { dataUrl, name }

const plusMenu = document.getElementById('plusMenu');
const menuUploadFiles = document.getElementById('menuUploadFiles');
const menuCreateImage = document.getElementById('menuCreateImage');

attachBtn.addEventListener('click', (e) => {
    e.stopPropagation();
    plusMenu.classList.toggle('open');
});

document.addEventListener('click', (e) => {
    if(!plusMenu.contains(e.target) && e.target !== attachBtn){
        plusMenu.classList.remove('open');
    }
});

menuUploadFiles.addEventListener('click', () => {
    plusMenu.classList.remove('open');
    fileInput.click();
});

const modeChipRow = document.getElementById('modeChipRow');
let imageMode = false;

function setImageMode(on){
    imageMode = on;
    modeChipRow.innerHTML = '';
    if(on){
        inputEl.placeholder = 'Describe an image...';
        const chip = document.createElement('div');
        chip.className = 'mode-chip';
        chip.innerHTML = `<span>■ Creating image</span><span class="x" title="Cancel">X</span>`;
        chip.querySelector('.x').onclick = () => setImageMode(false);
        modeChipRow.appendChild(chip);
    } else {
        inputEl.placeholder = 'Ask B-T AI';
    }
    inputEl.focus();
}

menuCreateImage.addEventListener('click', () => {
    plusMenu.classList.remove('open');
    setImageMode(true);
});

fileInput.addEventListener('change', () => {
    const file = fileInput.files[0];
    if(!file) return;
    const reader = new FileReader();
    reader.onload = () => {
        pendingAttachment = { dataUrl: reader.result, name: file.name };
        renderAttachPreview();
    };
    reader.readAsDataURL(file);
    fileInput.value = '';
});

function renderAttachPreview(){
    attachPreview.innerHTML = '';
    updateSendButtonIcon();
    if(!pendingAttachment) return;
    const chip = document.createElement('div');
    chip.className = 'attach-chip';
    chip.innerHTML = `<span>${pendingAttachment.name}</span><span
class="remove" title="Remove">X</span>`;
    chip.querySelector('.remove').onclick = () => { pendingAttachment = null; renderAttachPreview(); };

```

```

    attachPreview.appendChild(chip);
  }

  // -----
  // Voice input (mic button) – uses the browser's built-in
  // Speech Recognition, free, no API key needed.
  // -----
  const SpeechRecognitionAPI = window.SpeechRecognition || window.webkitSpeechRecognition;
  let recognition = null;
  let isListening = false;
  const voiceIndicator = document.getElementById('voiceIndicator');

  if(SpeechRecognitionAPI){
    recognition = new SpeechRecognitionAPI();
    recognition.continuous = false;
    recognition.interimResults = false;
    recognition.lang = 'en-US';

    recognition.onresult = (event) => {
      const transcript = event.results[0][0].transcript;
      inputEl.value = (inputEl.value ? inputEl.value + ' ' : '') + transcript;
      inputEl.dispatchEvent(new Event('input'));
    };
    recognition.onend = () => {
      isListening = false;
      micBtn.classList.remove('mic-active');
      voiceIndicator.classList.remove('active');
    };
    recognition.onerror = () => {
      isListening = false;
      micBtn.classList.remove('mic-active');
      voiceIndicator.classList.remove('active');
    };
    micBtn.addEventListener('click', () => {
      if(isListening){
        recognition.stop();
        return;
      }
      isListening = true;
      micBtn.classList.add('mic-active');
      voiceIndicator.classList.add('active');
      recognition.start();
    });
  } else {
    micBtn.addEventListener('click', () => {
      alert('Voice input is not supported in this browser. Please try Chrome.');
```

```

const cleanedPrompt = cleanImagePrompt(promptText);

let thread = state.threads.find(t => t.id === state.activeId);
if(!thread){
  thread = { id: crypto.randomUUID(), title: promptText.slice(0, 40), messages: [] };
  state.threads.push(thread);
  state.activeId = thread.id;
}
if(thread.messages.length === 0){ thread.title = promptText.slice(0, 40); }

thread.messages.push({ role: 'user', content: promptText, timestamp: Date.now() });
save();
renderAll();

const typingWrap = document.createElement('div');
typingWrap.className = 'msg-wrap ai';
typingWrap.innerHTML = `<div class="bubble"><div class="typing"><span></span><span></span><span></span><span></span><span></span></div></div>`;
messagesEl.appendChild(typingWrap);
messagesEl.scrollTop = messagesEl.scrollHeight;

let imageUrl = null;
try{
  const res = await fetch(`${BACKEND_URL}/api/generate-image`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ prompt: cleanedPrompt })
  });
  const data = await res.json();
  imageUrl = data.imageUrl || null;
}catch(err){
  console.error('Image generation failed:', err);
}

typingWrap.remove();
thread.messages.push({
  role: 'assistant',
  type: 'image',
  content: imageUrl ? cleanedPrompt : 'Image nahi ban saki. Dobara koshish karein.',
  imageUrl,
  timestamp: Date.now()
});
state.usedToday++;
save(); saveUsage();
renderAll();
}

async function retryMessage(assistantIdx){
  const thread = state.threads.find(t => t.id === state.activeId);
  if(!thread) return;
  const userMsg = thread.messages[assistantIdx - 1];
  if(!userMsg || userMsg.role !== 'user') return;

  thread.messages.splice(assistantIdx, 1);
  save();
  renderAll();

  const typingWrap = document.createElement('div');
  typingWrap.className = 'msg-wrap ai';
  typingWrap.innerHTML = `<div class="bubble"><div class="typing"><span></span><span></span><span></span><span></span><span></span></div></div>`;
  messagesEl.appendChild(typingWrap);
  messagesEl.scrollTop = messagesEl.scrollHeight;

  try{
    const reply = await getAIResponse(userMsg.content || 'Describe this image.');
```

```

async function sendMessage(){
  const text = inputEl.value.trim();
  const attachment = pendingAttachment;
  if(!text && !attachment) return;

  inputEl.value = '';
  inputEl.style.height = 'auto';
  pendingAttachment = null;
  renderAttachPreview();
  sendBtn.disabled = true;

  // If the message looks like an image request, generate an image instead
  if(imageMode && text){
    setImageMode(false);
    await handleImageRequest(text);
    return;
  }

  // Auto-detect from plain typed text too (e.g. "image banao a sunset")
  if(text && isImageRequest(text)){
    await handleImageRequest(text);
    return;
  }

  if(state.usedToday >= DAILY_LIMIT){
    alert("You've reached today's free limit (100 messages). Upgrade to Pro for unlimited access.");
    return;
  }

  let thread = state.threads.find(t => t.id === state.activeId);
  if(!thread){
    thread = { id: crypto.randomUUID(), title: (text || 'Image').slice(0, 40), messages: [] };
    state.threads.push(thread);
    state.activeId = thread.id;
  }
  if(thread.messages.length === 0){ thread.title = (text || 'Image').slice(0, 40); }

  thread.messages.push({ role: 'user', content: text, attachedImage: attachment ? attachment.dataUrl : null, timestamp: Date.now() });
  save();
  renderAll();

  const typingWrap = document.createElement('div');
  typingWrap.className = 'msg-wrap ai';
  typingWrap.innerHTML = `<div class="bubble"><div class="typing"><span></span><span></span><span></span><span></span><span></span></div></div>`;
  messagesEl.appendChild(typingWrap);
  messagesEl.scrollTop = messagesEl.scrollHeight;

  showStopButton();
  try{
    const reply = await getAIResponse(text || 'Describe this image. ');
    typingWrap.remove();
    thread.messages.push({ role: 'assistant', content: reply, timestamp: Date.now() });
  } catch(err){
    typingWrap.remove();
    if(err.name === 'AbortError'){
      thread.messages.push({ role: 'assistant', content: 'Stopped generating.', timestamp: Date.now() });
    } else {
      thread.messages.push({
        role: 'assistant',
        content: "Sorry, I couldn't reach the server. Please check your connection and try again.",
        timestamp: Date.now(),
        failed: true
      });
    }
  }
  hideStopButton();
  state.usedToday++;
  save(); saveUsage();
  renderAll();
}

// -----
// Replace this function to call YOUR backend (see server.js),
// which securely calls the AI provider using your own API key.
// Never call the AI API directly from this frontend in production –
// that would expose your API key to every visitor.
// -----
let activeController = null;

```



```

function showStopButton(){
  sendBtn.classList.add('stop-mode');
  sendBtn.disabled = false;
  sendBtn.innerHTML = `<svg viewBox="0 0 24 24" fill="currentColor"><rect x="6" y="6" width="12" height="12" rx
="2"/></svg>`;
}
function hideStopButton(){
  sendBtn.classList.remove('stop-mode');
  updateSendButtonIcon();
}

async function getAIResponse(userText){
  activeController = new AbortController();
  try{
    const res = await fetch(`${BACKEND_URL}/api/chat`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ conversationId: state.activeId, message: userText, userId: currentUserId }),
      signal: activeController.signal
    });
    if(!res.ok) throw new Error('backend not available');
    const data = await res.json();
    return data.reply;
  } finally {
    activeController = null;
  }
}

// -----
// Full-screen voice call mode – like ChatGPT/Gemini's voice
// conversation: listen, send to AI, speak the reply, repeat.
// Uses the browser's built-in Speech Recognition + Speech
// Synthesis (free, no extra API).
// -----
const callScreen = document.getElementById('callScreen');
const callStatus = document.getElementById('callStatus');
const callEndBtn = document.getElementById('callEndBtn');
const callMuteBtn = document.getElementById('callMuteBtn');
let callActive = false;
let callMuted = false;
let callRecognition = null;

// If running inside the B-T AI Android app, AndroidVoice is injected by
// MainActivity.kt and gives us native speech recognition + text-to-speech,
// since Android's WebView doesn't support the browser's Web Speech API.
const hasNativeVoice = !!window.AndroidVoice;

window.onNativeSpeechResult = function(text){
  if(callActive && text) callHandleTurn(text);
  else if(callActive) setTimeout(callListenTurn, 600);
};
window.onNativeSpeechError = function(){
  if(callActive && !callMuted) setTimeout(callListenTurn, 600);
};
window.onNativeSpeechDone = function(){
  if(callActive) callListenTurn();
};

function setCallState(state){
  callScreen.classList.remove('listening', 'thinking', 'speaking');
  callScreen.classList.add(state);
  const labels = { listening: 'Listening...', thinking: 'Thinking...', speaking: 'Speaking...' };
  callStatus.textContent = labels[state] || '';
}

function stripMarkdownForSpeech(text){
  return (text || '')
    .replace(/``[\s\S]*?``/g, '')
    .replace(/`([^\`]+)`/g, '$1')
    .replace(/\*\*([^\*]+)\*\*/g, '$1')
    .replace(/\^[^*]+\$/gm, '');
}

function openCallMode(){
  if(!hasNativeVoice && (!SpeechRecognitionAPI || !window.speechSynthesis)){
    alert('Voice call is not supported in this browser. Please try Chrome.');
```

```

    callMuteBtn.classList.remove('muted');
    callScreen.classList.add('open');
    callListenTurn();
}

function closeCallMode(){
  callActive = false;
  if(hasNativeVoice){
    window.AndroidVoice.stopListening();
    window.AndroidVoice.stopSpeaking();
  } else {
    if(callRecognition){ try{ callRecognition.abort(); }catch(e){} }
    window.speechSynthesis.cancel();
  }
  callScreen.classList.remove('open', 'listening', 'thinking', 'speaking');
}

callEndBtn.addEventListener('click', closeCallMode);
callMuteBtn.addEventListener('click', () => {
  callMuted = !callMuted;
  callMuteBtn.classList.toggle('muted', callMuted);
  if(callMuted && callRecognition){ try{ callRecognition.abort(); }catch(e){} }
  else if(callActive) callListenTurn();
});

function callListenTurn(){
  if(!callActive || callMuted) return;
  setCallState('listening');

  if(hasNativeVoice){
    window.AndroidVoice.startListening();
    return;
  }

  callRecognition = new SpeechRecognitionAPI();
  callRecognition.continuous = false;
  callRecognition.interimResults = false;
  callRecognition.lang = 'en-US';

  callRecognition.onresult = (event) => {
    const transcript = event.results[0][0].transcript;
    callHandleTurn(transcript);
  };
  callRecognition.onerror = () => {
    if(callActive && !callMuted) setTimeout(callListenTurn, 600);
  };
  callRecognition.onend = () => {};
  callRecognition.start();
}

async function callHandleTurn(transcript){
  if(!callActive) return;
  setCallState('thinking');

  let thread = state.threads.find(t => t.id === state.activeId);
  if(!thread){
    thread = { id: crypto.randomUUID(), title: transcript.slice(0, 40), messages: [] };
    state.threads.push(thread);
    state.activeId = thread.id;
  }
  thread.messages.push({ role: 'user', content: transcript, timestamp: Date.now() });
  save();

  let reply;
  try{
    reply = await getAIResponse(transcript);
  } catch(err){
    reply = "Sorry, I couldn't reach the server.";
  }
  thread.messages.push({ role: 'assistant', content: reply, timestamp: Date.now() });
  state.usedToday++;
  save(); saveUsage();
  if(state.activeId === thread.id) renderAll();

  if(!callActive) return;
  setCallState('speaking');
  const speechText = stripMarkdownForSpeech(reply);

  if(hasNativeVoice){
    window.AndroidVoice.speak(speechText);
    return;
  }

```

```
    }

    const utterance = new SpeechSynthesisUtterance(speechText);
    utterance.onend = () => {
      if(callActive) callListenTurn();
    };
    utterance.onerror = () => {
      if(callActive) callListenTurn();
    };
    window.speechSynthesis.speak(utterance);
  }

  renderAll();
</script>
</body>
</html>
```